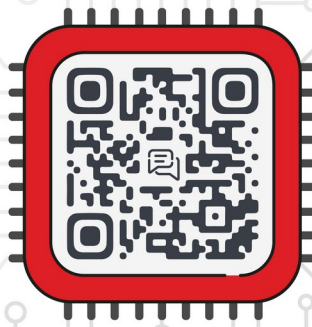




3
5
600 sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة - النظم والشبكات

Multi Threaded
Servers



RB Informatics; محتوى مجاني غير مخصص للبيع التجاري
17/11/2025

برمجة تطبيقات شبكية

السلام عليكم ورحمة الله وبركاته

عندما يدخل الـ Socket بحالة Block بانتظار رد من الـ client
فإن هذا يؤدي إلى تجميد كل الـ Threads الأخرى، لأن السيرفر يخدم:

one client per Socket

لكن ذلك على مستوى الأداء يعد سيئ جداً، لذلك نريد تحسين الأداء وذلك بعمل Multi threaded Server .
وهو سيرفر يستطيع التعامل مع عدة clients في نفس الوقت، حيث أن كل client يحصل على Thread خاص به،
حيث يصبح بإمكان السيرفر معالجة عدة عملاء بالتوازي دون انتظار انتهاء أحدهم لبدء الآخر بالعمل.

إدارة اتصالات الـ TCP في :

Operating System :

عندما تأتي عدة اتصالات TCP لنظام التشغيل فإنه يحتفظ بالاتصالات ويدير تنفيذها بواسطة (queueing mechanisms) أي يحتفظ بها في قائمة انتظار قبل أن يقرأها التطبيق.
ويتم رفض محاولات الاتصال التي تأتي بعد أن تمتلئ قائمة الانتظار الخاصة ببورت محدد.
هذا يمنع الخادم من الانهيار عند استقبال عدد كبير جداً من الطلبات في نفس اللحظة.

Java :

إن الـ Backlog الخاص بسوكيت السيرفر يتحكم بأكبر عدد طلبات يمكن أن تُحجز في قائمة الانتظار، وإن العدد الافتراضي يتم تحديده من قبل الجهة (مثلاً C أو Java) التي أنشأت هذه البيئة، عند تحديد البورت يتم إنشاء سوكيت السيرفر هكذا:

```
ServerSocket ss = new ServerSocket(port);
```

BackLog :

هو قائمة مهام بانتظار الإنجاز، توضع في Buffer التي ينشئها الـ Socket عند فتحه على الجهاز، وفيها أقصى عدد من الاتصالات established but not accept التي يمكن أن يحتفظ بها النظام.

إن أفضل عمل يؤدي لرفع الأداء هو عند تحديد الـ backlog حسب البورت، أي:

```
ServerSocket ss = new ServerSocket(port, backlog);
```

أي يتم تحديد الـ backlog أثناء برمجة السيرفر، لكن السيرفرات التكرارية غير عملية في العديد من الأوقات، فيكون الحل:

• Thread per connection (Multithreading)

• Thread pool

لماذا غير عملية؟

إذا استغرق طلب عميل واحد وقتاً طويلاً جداً (ربما لأنه ينتظر إدخال/إخراج بطيء مثل "slow output" أو قراءة من القرص)، فإن الطلبات الأخرى تتراكم في طابور backlog الخاص بنظام التشغيل ("full queue") وتحرم من الخدمة، مما يؤدي إلى عدم استجابة الخادم.

: Thread per connection (Multithreading)

بمجرد أن يقبل السيرفر طلباً من ال client يُنشئ له Thread خاص به، فإذا حدث تأخير في أحدهم لا يؤثر على الباقي.

: Thread pool

هنا يقوم السيرفر بإنشاء Threads مسبقاً قبل وصول الطلبات (عند تشغيله) ويجعلها في حالة انتظار، ويضع الطلبات في queue، وأي Thread فارغ نخصه لطلب من لا queue (first in first out)، وإذا كان قد وصل عدد الطلبات للحد (أي الطلبات شغلت كل ال Threads المجهزة سابقاً) ستبقى الطلبات الأخرى في حالة انتظار لحين أن تتحرر أحد ال Threads.

Thread per connection design

كل اتصال جديد من client نعطيه Thread منفصل خاص به، ويبقى ال Thread الأساسي الذي أنشأناه فقط ليقبل الاتصالات (accept).

Example 1: Multi Threaded

السيرفر متصل على البورت (13) في حالة listening للاتصالات، كلما اتصل client جديد يقبل السيرفر الاتصال وينشئ Thread جديد.

كل Thread يرسل الوقت والتاريخ لل client الخاص به ثم يغلق الاتصال، وهكذا يمكن للسيرفر استقبال أكثر من عميل في نفس الدقيقة.

1- أولاً ننشئ ServerSocket، ويُستخدم try-with-resources لأنه ثابت للسيرفر، ويُغلق عند انتهاء البرنامج.

2- نضع حلقة ليبقى في حالة انتظار عملاء، يقبلهم، وعندما يتصل client جديد يُنشئ Socket خاصاً به.

3- يتم إنشاء Thread جديد عند كل اتصال، حيث يرسل التاريخ والوقت لكل client خاص به.

4- هنا يتم تعريف الكلاس الذي سيقوم بعمل ال Threads، حيث يأخذ ال Socket الخاص بال client ويفتح OutputStream للكتابة عليه.

5- يرسل الوقت والتاريخ ويُغلق ال Socket. يتم هنا

الإغلاق داخل ال Thread في كتلة finally، لأنه إذا تم إغلاق السوكيت عند انتهاء بلوك ال try ولم يكن ال Thread قد أنهى عمله، سيؤدي ذلك لفشله.

```
public class MultithreadedDaytimeServer {
    public static void main(String[] args) {
        //1
        try (ServerSocket server = new ServerSocket(13)) {
            //2
            while (true) {
                try {
                    Socket connection = server.accept();
                    //3
                    Thread task = new DaytimeThread(connection);
                    task.start();

                } catch (IOException ex) {}
            }
        } catch (IOException ex) {
            System.err.println("Couldn't start server");
        }
    }
    //4
    private static class DaytimeThread extends Thread {
        private Socket connection;

        DaytimeThread(Socket connection) {
            this.connection = connection;
        }

        @Override
        public void run() {
            try {
                //5
                Writer out = new OutputStreamWriter(connection.getOutputStream());
                Date now = new Date();
                out.write(now.toString() + "\r\n");
                out.flush();

            } catch (IOException ex) {
                System.err.println(ex);
            } finally {
                try {
                    connection.close();
                } catch (IOException e) {}
            }
        }
    }
}
```

Virtual Threads

Threads	Virtual Threads
كلفة إنشاؤه عالية حيث يحتاج ذاكرة لل Stack تقريبا 1 MB	- تكلفة : إنشاؤه منخفضة حيث أنها تقريبا 100 KB
تتم ادارته من قبل نظام التشغيل (OS)	تتم ادارته من قبل ال JVM
يمكن إنشاء عدد محدود منها فهي محدودة بموارد الجهاز وقدرته الى cpu	يمكن إنشاء عدد كبير جدا منها وهي محدودة بذاكره ال JVM
	عندما يحصل Block لا يتم إعاقه تنفيذ ال OS Threads

Example 2:

```

public class MultithreadedDaytimeServer {
    public static void main(String[] args) {
        //1
        try (ServerSocket server = new ServerSocket(13)) {
            while (true) {
                try {
                    //2
                    Socket connection = server.accept();
                    //3
                    Thread task = Thread.ofVirtual().start(new Runnable() {
                        @Override
                        public void run() {
                            handleDaytimeRequest(connection);
                        }
                    });
                } catch (IOException ex) {
                    System.err.println("Accept error: " + ex.getMessage());
                }
            }
        } catch (IOException ex) {
            System.err.println("Couldn't start server: " + ex.getMessage());
        }
    }
    //4
    private static void handleDaytimeRequest(final Socket connection) {
        try (Socket conn = connection;
            Writer out = new OutputStreamWriter(conn.getOutputStream())) {
            Date now = new Date();
            out.write(now.toString() + "\r\n");
            out.flush();

        } catch (IOException ex) {
            System.err.println("Client handling error: " + ex.getMessage());
        }
        // No explicit finally needed - try-with-resources handles closing
    }
}

```

1- تم فتح ServerSocket على البورت 13.

2- يتم قبول اتصال من العميل باستخدام accept(), فيقبل اتصال من ال Backlog ويُعيد Socket تمثل هذا الاتصال.

3- ينشئ ويشغل Virtual Thread جديدة، وداخل ال run() تستدعي الدالة handleDaytimeRequest(writer) من نوع writer.

4- هذه الدالة تتعامل مع ال client، تفتح Writer من نوع OutputStream، ثم وضع السوكت الخاص بالاتصال ضمن try-with-resources لضمان أن يُغلق ال Socket عند الخروج من البلوك حتى لو حدث استثناء أثناء التنفيذ.

لماذا وضعنا try-with-resources داخل الدالة وليس في ال main كما قبل؟
لأنه إذا وضعناها في ال main سيتم غلق ال Socket عند انتهاء ال main، وقد لا يكون ال Thread قد أنهى عمله بعد.

Thread per connection Problem

في هذه الطريقة كما رأينا، عندما يطلب الـ client اتصالاً يتم عندها إنشاء Thread. لكن ماذا لو كانت عدد الاتصالات هائلة في وقت واحد؟ سيؤدي ذلك لاستهلاك ذاكرة الـ RAM، لأنه لو طلب مثلاً 1000 اتصال (أو أكثر) وكل اتصال ينشئ له السيرفر Thread بحجم 1MB تقريباً، فسيسبب ذلك أيضاً استنزافاً للـ CPU، وبعدها قد تنهار الـ JVM. تُسمى هذه الحالة (Denial of Service Attack (DoS).

لذلك تم إيجاد حل هو :

Thread pool

حيث أنه عند إنشاء السوكت يتم إنشاء عدد محدود من الـ Threads (Thread pool). يتم وضع الاتصالات الواردة ضمن queue، ويتم تخديمها من قبل الـ Threads المتاحة ضمن الـ pool. في هذه الحالة نضمن أن استخدام الذاكرة ثابت، وأداء الـ CPU جيد، والسيرفر يبقى مستقراً دون إنشاء Threads جديدة دائماً.

Executor Service

هي الأداة التي توفر الأداء العالي في Java، ضمن الحزمة java.util.concurrent. وظائفها:

- إنشاء Thread pool
 - تنفيذ المهام بشكل متزامن على Threads
 - إدارة queue وكيفية تخديم الـ connection حسب الأولوية
 - تسهيل إيقاف الـ Threads بطريقة منظمة
- كيفية إنشاء Thread pool باستخدام Executor Service:**

```
import java.util.concurrent.ExecutorService
import java.util.concurrent.Executors

ExecutorService pool = Executors.newFixedThreadPool(10)
```

هنا تم إنشاء pool يحتوي على 10 Threads. يمكن تعيين عدد Threads بإرسال عدد ثابت إلى newFixedThreadPool. لتنفيذ المهام:

```
pool.submit(Runnable task)
pool.execute(Runnable task)
```

لإيقاف الـ Thread pool:

```
pool.shutdown(); // يتوقف بعد انتهاء المهام الحالية
pool.shutdownNow(); // يتوقف حالا دون انتظار انتهاء المهام الحالية
```

Multi Threaded Servers using Thread pool

- 1- تنشئ Thread pool.
- 2- عند وصول اتصال من client جديد يُضاف إلى قائمة الانتظار على أنه task، وعند توفر Thread فارغ من الـ pool يتم تعيينه له.
- 3- يصبح الـ Thread متاحًا لاتصال آخر بعد أن ينتهي من معالجة الاتصال الحالي.

Example:

```
import java.io.*;
import java.net.*;
import java.util.*;
import java.util.concurrent.*;

public class PooledDaytimeServer {
    public static void main(String[] args) {
        //1
        ExecutorService pool = Executors.newFixedThreadPool(50);

        //2
        try (ServerSocket server = new ServerSocket(13)) { //start of 1st try
            while (true) {
                try { //start of 2nd try

                    //3
                    Socket connection = server.accept();
                    Runnable task = new DaytimeTask(connection);
                    pool.submit(task);
                } catch (IOException ex) {} //end of 2nd try
            } // end of while
        } catch (IOException ex) {} //end of 1st try
        System.err.println("Couldn't start server");

        //6
        pool.shutdown();
    }

    //4
    private static class DaytimeTask implements Runnable {
        private Socket connection;
        DaytimeTask(Socket connection) {
            this.connection = connection;
        }
        @Override
        public void run() {
            try {
                Writer out = new OutputStreamWriter(connection.getOutputStream());
                Date now = new Date();
                out.write(now.toString() + "\r\n");

                //5
                out.flush();
            } catch (IOException ex) {
                System.err.println(ex);
            } finally {
                try {
                    connection.close();
                } catch (IOException e) {} // ignore;
            }
        }
    }
}
```

1- إنشاء Thread pool يحتوي على 50 Threads.

2- إنشاء سوكيت السيرفر ويكون في حالة listening لاستقبال وقبول الطلبات.

3- إرسال مهمة لأحد الـ Threads ضمن الـ pool.

4- تعريف كلاس DayTimeTask الذي يقوم بالتعامل مع اتصال العميل و Socket الخاص به، وينفذ المهمة على أي Thread متاح، وينشئ stream لإرسال البيانات للعميل.

5- لضمان إرسال البيانات مباشرة، ثم إغلاق اتصال العميل.

6- إغلاق Thread pool عند الانتهاء.

THE END